CAPFUND WALLET APP – TECHNICAL DOCUMENTATION

I. BACKGROUND

Capataz' solutions help clients reach their goals. It assists the clients in managing their business more efficiently, reducing manual effort, and increasing convenience—automating many parts of their operations so they can focus on scaling up.

CapFund is a natural extension of this mission. It is a secure, QR-based digital wallet system designed to streamline internal employee transactions. By supporting monthly reloadable balances, real-time transaction tracking, and centralized kiosk management, the app reduces administrative overhead and enhances employee convenience. This initiative aligns with Capataz' vision of digital transformation—empowering businesses to operate smarter, faster, and with greater transparency.

II. OBJECTIVES

- Provide employees with a cashless, secure, and convenient payment method.
- Enable real-time financial tracking and reporting for administrators.
- Automate monthly balance resets and reduce manual financial processes.
- Integrate seamlessly with existing Capataz workforce management tools.

III.SCOPE OF IMPACT

	Employee Wallet	Kiosk Interface	Admin Dashboard	Backend Services	Database
Employees	✓	X	X	X	X
Kiosk Operators	X	✓	X	X	X
HR/Admin Staff	X	X	✓	X	X
IT/DevOps Team	X	X	X	✓	✓
Finance/Accounting	X	X	✓	X	✓

Table 1. Scope of Impact

A. Impacted Modules

- Employee Wallet (Mobile App): QR generation, balance display, transaction history, push notifications.
- Kiosk Interface: Transaction processing, QR scanning, and category selection.
- Admin Dashboard: Wallet configuration, kiosk management, reporting.
- Backend Services: APIs for transactions, user management, and reporting.
- Database: New schemas for wallet balances, transactions, kiosk metadata.

B. Impacted Stakeholders

- Employees
- Kiosk Operators
- HR/Admin Staff
- IT/DevOps Teams
- Finance/Accounting

C. Functional Impact

- QR-based payments with dynamic refresh
- Real-time balance updates and notifications
- Exportable transaction summaries
- Monthly auto-reset of balances
- Kiosk category filtering and management

IV. FEATURES AND LIMITATIONS

A. Frontend Features

Employee

- View Current Balance: Employees can see their wallet balance in real time.
- View Transaction History: Employees can view all their transactions, filtered by their wallet ID.
- Generate QR Codes: Employees can generate QR codes for transactions (e.g., for kiosk payments).

Admin Features

- View All Transactions: Admins can view and filter all company transactions.
- Advanced Filtering: Filter transactions by employee, kiosk, category, type, status, and date.
- User Management: Admins can add, edit, and manage employees and kiosk operators.
- Kiosk Management: Admins can add, edit, and manage kiosks.
- Company Configuration: Admins can manage company settings and configurations.
- Reporting: Generate and export reports (PDF, CSV, XLSX) with custom date ranges and filters.
- Notifications: In-app and push notifications for relevant events.

General Features

- Authentication: Secure login with JWT token storage and session management.
- Responsive UI: Built with Ionic for mobile and desktop compatibility.
- RESTful API Integration: All data is fetched and updated via REST API endpoints.
- Modular Structure: Clear separation of concerns with core services, shared components, and feature modules.
- Environment Configuration: API URLs and settings managed via environment files.
- Extensible: Easy to add new pages, services, and features.

B. Backend Features

Framework & Architecture

- **.NET 9 Web API** - Built with ASP.NET Core, targeting .NET 9 to take advantage of modern performance enhancements.
- **Layered Architecture** - Promotes separation of concerns with a clean and organized structure:
    - src/Application – Contains business logic, CQRS using MediatR, DTOs, and validation.
    - src/Domain – Holds domain entities, enums, and core business rules.
    - src/Infrastructure – Includes data access using Entity Framework Core, external service integrations, and dependency injection.

- src/Web – Defines API endpoints, middleware, application configuration, and startup logic.

Database Integration

- Uses Entity Framework Core as the ORM.
- Supports AWS RDS SQL Server with flexible connection string configuration.
- Database migrations are managed via EF Core CLI.

Authentication & Authorization

- Custom authentication system with JWT tokens
- Role-based authorization is implemented using [Authorize(Roles = "...")] attributes.

API Documentation

- Utilizes NSwag for generating OpenAPI/Swagger documentation.
- Swagger UI is available at the /api route for easy exploration and testing of API endpoints.

Background Processing

- Hangfire is integrated for managing background jobs.
- Includes a dashboard for monitoring job execution and history.

Real-Time Communication

- SignalR hub configured at /transactionHub for pushing real-time updates to clients.

Health Monitoring

- Health checks are exposed through a dedicated endpoint at /health.

Cross-Origin Resource Sharing (CORS)

- CORS policy is configured to allow cross-origin requests.
- The current policy (AllowAllOrigins) can be adjusted based on the deployment environment.

Error Handling

- A global exception handler is configured to manage unhandled errors and provide consistent API error responses.

Configuration Management

- Supports environment-based configuration files:
    - appsettings.json
    - Appsettings.Development.json

Reports and File Handling

- Provides endpoints to generate and download reports.
- Configurable local folder path used to store generated report files.

V. FUNCTIONAL FEATURES OVERVIEW

A. Included Functional Features

Wallet & Balance

- Users can view their wallet balance based on credit limit.
- Users can see how much credit has been used and how much is available.

Note: On the employee side, only the real balance is shown, not the credit limit.

Payments

- Users receive payment confirmation after successful or failed transactions.

Admins can:

- View the complete transaction history.
- Filter transactions by type (e.g., reset, manual adjustment, purchase).
- Search for specific transactions.

Spending Insights & Budget

- Users receive notifications for all key activities, including payments, top-ups, and requests.

Admins can:

- View charts and analytics showing how much has been spent by category.

B. Future Functional Features (Not Included)

Wallet & Balance

- Users cannot currently top-up their wallet using bank or other payment channels.
- Users cannot view their top-up history.
- Monthly budget setting and alerts when approaching the budget limit are not available.

Payments

- Users cannot make payments by manually entering a mobile number or account ID.
- Users cannot confirm and review payment details before sending.

Send & Receive Money

- Sending money via mobile number, QR code, or email is not supported.
- Money request feature (send/track requests) is not available.

Spending Insights & Budget

Users cannot:

- View a summary of spending habits.
- Manage notification preferences (push, email, SMS).
- View a list of past notifications in-app.

Account & Security

- Secure login options (PIN, fingerprint, face recognition) are not implemented.
- Password or PIN change functionality is not available.
- Users cannot log out remotely from all devices.
- Login activity tracking is not available.

Linked Accounts

- Users cannot link or manage multiple bank accounts or cards.
- Users cannot choose a default payment source.
- Users cannot set spending limits per linked account.

Support & Help

- No in-app help center or FAQ is available.
- Users cannot contact support via chat or email.
- Reporting issues for specific transactions is not supported.

Promotions & Rewards

Users cannot:

- View active promotions or cashback offers.
- Redeem promo codes or vouchers.
- Track cashback earnings or promotion usage.

VI. SYSTEM LIMITATIONS

These are known current limitations of the system and areas where additional features or enhancements may be required.

A. Real-Time and Offline Functionality

Limited Real-Time Sync

- Only select features use real-time technologies (e.g., SignalR for transaction group join). Most data are fetched on demand.

Limited Offline Support

- The system relies on live API calls. There is minimal to no functionality available when offline.

B. User Interface and Experience

No Role-Based UI Restrictions in Frontend

- While role-based access control exists, the frontend does not fully restrict UI elements. Protection relies primarily on backend enforcement.

No Internationalization (i18n)

- The application is only available in English. Support for multiple languages is not implemented.

No Accessibility (a11y) Review

- There is no documented accessibility compliance or support for users with disabilities.

C. Reporting and Analytics

No Deep Analytics

- Reporting features are limited to basic filters and data exports. Advanced dashboards and analytics are not included.

D. Payments and Notifications

No Payment Gateway Integration

- The system does not integrate with external payment providers. All transactions are handled internally.

No Built-in Email or SMS Services

- There is no evidence of integrated messaging services for user notifications via email or SMS.

E. Security and Account Management

No User Self-Registration

- Account registration may require admin intervention or backend support. There is no user-driven registration flow.

Limited Role Management

- While role-based authorization exists, advanced features like dynamic role assignment or a permissions matrix are not included.

F. Testing and Error Handling

No Automated Testing

- There is no documentation or indication of unit, integration, or end-to-end automated tests.

Limited Error Handling

- Error handling appears basic, with limited support for detailed user-facing error messages or troubleshooting guidance.

G. System Architecture and Scalability

No Built-in Monitoring/Telemetry

- There is no integration with monitoring tools like Application Insights or other telemetry systems.

No Built-in Rate Limiting or API Throttling

- The system lacks features to control abuse or excessive usage via API rate limiting.

No Scalability or Multi-Tenancy Support

- Features such as sharding, distributed caching, or tenant separation are not mentioned.

H. Data and Storage

No Built-in Data Seeding or Demo Data

- There is no mechanism to seed initial data for development, testing, or demos.

No Built-in File Storage Abstraction

- Reports are saved to a local folder. There is no abstraction layer to support cloud storage

VII. TECHNICAL IMPACT

A. Frontend

- Mobile App: Built with Ionic + Angular
- Kiosk Interface: Angular based with QR scanner integration
- Admin Dashboard: Mobile dashboard for configuration and reporting

B. Backend

- ASP.NET Web API with JWT authentication
- Integration with MSSQL database
- AES-256 encryption for QR codes
- Push notifications via Firebase/OneSignal

C. Dependencies

- Capataz HRIS and payroll systems
- Cloud hosting: AWS EC2 for application hosting and AWS RDS for database services.
- Firebase/OneSignal for notifications
- QR scanner plugins and mobile app stores

VIII. RISKS AND MITIGATION

Risk	Impact	Mitigation
QR code spoofing	High	Use encrypted, time-sensitive QR codes

Data loss	High	Implement regular backups and failover systems
User confusion	Medium	Provide onboarding and in-app tutorials
Kiosk downtime	Medium	Enable kiosk status monitoring and fallback options

Table 2. Risks and Mitigation

IX. TESTING AND VALIDATION

- Unit Testing: Backend APIs and frontend components
- Integration Testing: Across employee, kiosk, and admin modules
- User Acceptance Testing (UAT): Based on defined user stories and acceptance criteria
- Performance Testing: Transaction speed and load handling

X. TIMELINE AND MILESTONES

Phase	**Description**	**Status**
Phase 1: Frontend Design	UI/UX design for Employee Wallet, Kiosk Interface, and Admin Dashboard (Figma)	Completed
Phase 2: Workplan Finalization	Defined deliverables, sprint goals, and team responsibilities	Completed

Phase 3: Frontend & Backend Development	Developed mobile app, kiosk interface, admin dashboard, and backend APIs	Completed
Phase 4: Integration & Testing	Integrated modules and conducted functional, UAT, and performance testing	Completed
Phase 5: Technical Documentation	Preparing API references, user guides, and system architecture documentation	Completed
Phase 6: Deployment	Final deployment to cloud platforms and app stores	Completed

Table 3. Timeline and Milestones

XI. CONCLUSION

The Capataz Employee Wallet App is a strategic initiative that enhances employee convenience, reduces administrative overhead, and strengthens financial transparency. With a clear roadmap, robust architecture, and user-centered design, the app is poised to deliver significant value to both employees and administrators.

FRONTEND

A. Technology Stack:

- Angular (with Ionic Framework)
- Ionic (UI components and mobile support)
- TypeScript
- RxJS
- Angular Forms (FormsModule, ReactiveFormsModule)
- Angular Animations
- Angularx-qrcode (for QR code generation)
- Capacitor (for native device features)
    - @capacitor/push-notifcations
    - @capacitor/local-notifications
    - @capacitor/status-bar

B. Folder Structure

src/app/

core/ // Core services (authentication, transactions, etc.)

pages/ // Main application pages (views)

shared/ // Shared components (footer, transaction item, etc.)

models/ // TypeScript interfaces and models

assets/ // Static assets (icons, images)

environments/ // Environment configuration files

theme/ // SCSS theme variables

C. Key Modules and Components Pages

login: User authentication page.

register/role-options, register/step-one, register/step-two, register/step-three: User registration flow.

kiosk-ita: Kiosk interface for transactions.

kiosk-scan: Scan QR codes at kiosks.

generate-qr: Generate QR codes for transactions.

receipt: View and print transaction receipts.

notification: In-app notifications.

user-management: Admin management of users.

admin-dashboard: Admin dashboard overview.

admin-view-transactions**:** Admin interface for viewing and filtering all company transactions.

employee-view-transaction-history**:** Employee interface for viewing their own transaction history.

view-current-balance**:** Employee wallet and balance

view. company-config: Company configuration management (for admins).

D. Core Services

auth.service.ts: Handles authentication, user info, and token storage.

transaction.service.ts: Handles all transaction-related API calls.

employee-main-dashboard.service.ts: Fetches employee wallet and dashboard data.

notification.service.ts: Handles in-app notifications.

push-notification.service.ts: Handles push notifications.

kiosks-management.service.ts: Handles kiosk-related API calls.

company-config.service.ts: Handles company configuration.

category.service.ts: Handles category-related API calls.

E. Shared Components

footer.component.ts: Application footer.

transaction-item.component.ts: Displays a single transaction entry.

admin-menu-bar.component.ts: Admin navigation bar.

F. Main Application Flows

Authentication

- Users log in via the login page.
- JWT tokens and user info are stored in local storage.
- AuthService manages login, logout, and session checks.

Employee Flow

Employees can:

- View their current balance.
- View their transaction history (filtered by their wallet ID).
- Generate QR codes for transactions.

Admin Flow

Admins can:

- View and filter all transactions for their company.
- Manage employees, kiosks, and company settings.
- Filter transactions by employee, kiosk, category, type, status, and date.

G. API Communication

All data is fetched and updated via REST API endpoints.

Services use Angular's HttpClient to communicate with the backend.

H. Configuration

Environment Variables

API URLs and other environment-specific settings are stored in ***environment.ts.***

I. Build and Run:

Install dependencies: *npm install*

Run application: *ng serve*

Ionic: *ionic serve*

J. Adding New Features

- Add new pages under pages.
- Add new services under services.
- Register new components in the relevant module or as standalone.
- Update routing as needed in the Angular router configuration.

K. API Reference

Main Endpoints User	
User Authentication	/api/Users/login
Fetch, Filter, and Manage Transactions	/api/Transactions
Fetch Employee Data	/api/Employees
Fetch Kiosk Data	/api/Kiosks
Fetch Category Data	/api/Categories
Example: Fetching Transactions	***Endpoint: /api/Transactions***

Table 4. API Reference

L. Query Parameters:

WalletIds (array of integers)

TransactionTypes (array of strings)

TransactionStatuses (array of strings)

FromDate, ToDate (date strings)

CategoryNames (array of strings)

KioskIds (array of integers)

CompanyId (integer)

MonthRange, PageNumber, PageSize (integers)

M. Troubleshooting

Component Not Found: Ensure the component is imported in the @Component.imports array.

API Not Filtering: Double-check query parameter names (e.g., WalletIds vs. walletId).

Authentication Issues: Ensure tokens are stored and sent with each request.

N. Contributing

- Fork the repository and create a new branch for your feature or bugfix.
- Follow the existing code style and structure.
- Submit a pull request for review.

O. Glossary

- Access Point (AP): Device that provides wireless network access.
- Wallet ID: Unique identifier for an employee's wallet.
- Transaction: Record of a financial operation (purchase, reset, adjustment).
- Kiosk: Physical or virtual point where transactions can occur.

BACKEND

A. Overview

This backend is a modular, layered ASP.NET Core Web API targeting .NET 9. It is designed for scalability, maintainability, and integration with AWS RDS SQL Server. The project uses Entity Framework Core for data access, MediatR for CQRS, and NSwag for OpenAPI/Swagger documentation. It supports custom JWT-based authentication and includes Hangfire for background job processing

B. Solution Structure:

src/

Application/ # Application logic, CQRS handlers, DTOs, Validators

Domain/ # Domain entities, enums, and business rules

Infrastructure/ # Data access, external services, dependency injection

Web/ # API endpoints, middleware, configuration, startup

C. Key Components

1. **Web Project (src/Web)**
   - **Entry Point**: Program.cs
   - **Configuration**: appsettings.json, appsettings.Development.json
   - **Dependency Injection**: DependencyInjection.cs
   - **API Documentation**: config.nswag (NSwag for Swagger/OpenAPI)
   - **Endpoints**: Defined via MapEndpoints() and controllers or minimal APIs
   - **Real-time**: SignalR hub at /transactionHub
   - **Background Jobs**: Hangfire dashboard enabled

2. **Application Layer (src/Application)**
   - **CQRS**: Uses MediatR for commands and queries

- **DTOs**: Data Transfer Objects for API responses
- **Validation**: FluentValidation for request validation
- **Security**: Authorization attributes for endpoint protection
- **Business Logic**: Encapsulated in domain models

3. **Domain Layer (src/Domain)**
   - **Entities**: Core business entities (e.g., User, Employee, Kiosk, Transaction)
   - **Enums**: Status, device types, roles, etc.
4. **Infrastructure Layer (src/Infrastructure)**
   - **DbContext**: ApplicationDbContext using Entity Framework Core
   - **Database**: AWS RDS SQL Database (connection via DefaultConnection and/or CapFundDb)
   - **External Services**: Email, file storage, etc.
   - **Dependency Injection**: Service registrations

D. Configuration

- **Connection Strings**: Defined in appsettings.json and appsettings.Development.json under ConnectionStrings.
- **CORS**: Configured with AllowAllOrigins policy.
- **Swagger**: Available at /api for API exploration and testing.
- **Authentication**: AWS RDS SQL Server with flexible connection string configuration.

E. Database

- **Provider**: AWS RDS SQL Server
- **Migrations**: Managed via EF Core (dotnet ef database update)
- **Entities**: Employees, Users, Kiosks, Transactions, Reports, etc.

F. API Documentation

- **Swagger/OpenAPI**: Generated via NSwag (config.nswag). Accessible at /api.

G. Security

- **Authentication**: Integrated with custom authentication using JWT tokens
- **Authorization**: Role-based, using [Authorize(Roles = ...)] attributes

H. Background Jobs

- **Hangfire**: Used for background processing, dashboard available at /hangfire.

I. Real-Time Communication

- **SignalR**: Hub endpoint at /transactionHub for real-time updates.

J. Development & Deployment

- **.NET Version**: .NET 9
- **Local Developmen**t: Use appsettings.Development.json for local config.

K. Running the Project

1. Restore packages: dotnet restore
2. Apply migrations:

   dotnet ef database update --project src/Infrastructure --startup-project src/Web

3. Run the API: dotnet run --project src/Web
4. Access Swagger UI: https://localhost:{port}/api

L. Additional Notes

- **Logging**: Configurable in appsettings.json.
- **Reports**: Local folder path can be set in the Reports section of config.
- **Error Handling**: Global exception handler is configured.

References:

1. Capacitor Documentation. CapacitorJS Official Docs. Retrieved from https://capacitorjs.com/docs
2. Ionic Documentation. Ionic Framework Official Docs. Retrieved from https://ionicframework.com/docs
3. ASP.NET Core Documentation. Microsoft Learn. Retrieved from https://learn.microsoft.com/en-us/aspnet/core/?view=aspnetcore-9.0
4. Entity Framework Core Documentation. Microsoft Learn. Retrieved from https://learn.microsoft.com/en-us/ef/core/v
5. NSwag Documentation. Microsoft Learn & GitHub.
6. Microsoft Learn: https://learn.microsoft.com/en-us/aspnet/core/tutorials/getting-started-with-nswag?view=aspnetcore-8.0&tabs=visual-studio
   GitHub: https://github.com/RicoSuter/NSwag
7. Hangfire Documentation. Hangfire Official Site. Retrieved from https://www.hangfire.io/